

Stage 1 Lab - Human vs AI: Building Your First SmartCare Prototype

Learning objectives

- Create and run a simple Python file with basic input, output and processing statements
- Use lists, dictionaries and functions to enhance the Python file
- Build a small SmartCare appointment prototype.
- Use AI as a tutor rather than a replacement.
- Compare human-written and AI-generated code.
- Verify AI-generated code through execution and test inputs.
- Document a short AI-use reflection.

Files to create and commit in GitHub

stage01/
smartcare_v01.py
comparison.md
reflection.md
ai_usage.md

Part A - Understand the Problem: AI OFF

SmartCare needs a small prototype that allows a receptionist to record patient appointments. Each appointment records patient name, practitioner name and appointment time.

What data must be stored?

- Patient Name
- Doctor's name
- Appointment date and time

What functions might be useful?

- Adding a new appointment to the system
- Printing out all saved appointments and files
- Be able to search for existing records
- Delete a booking

What could go wrong?

- Two patients could be booked with the same doctor at the same time causing a double booking.
- A user could leave some fields to fill blank or input the wrong numbers for dates and times.
- All saved appointments will disappear when the script finishes running because data is only kept in temporary memory.

What requirements are unclear?

- How should dates and times be formatted or the ideal layout.
- Can a doctor handle multiple appointments at the same time or only one at a time.
- What permission does a user need to delete or edit for an appointment.

Part B - Build a Human-Written Prototype: AI OFF

#task 1

Create and run a simple Python file with basic input,output statements

```
print("Welcome to SmartCare: Community Clinic Appointment Booking System!")
```

First Appointment

```
patient1_name = 'Alice Smith'
```

```
practitioner1_name = 'Dr. John Doe'
```

```
appointment1_time = '2024-07-20 10:00 AM'
```

```
print(f"Patient: {patient1_name} | Practitioner: {practitioner1_name} | Time: {appointment1_time}")
```

Second Appointment

```
patient2_name = 'Bob Johnson'
```

```
practitioner2_name = 'Dr. Jane Roe'
```

```
appointment2_time = '2024-07-20 11:30 AM'
```

```
print(f"Patient: {patient2_name} | Practitioner: {practitioner2_name} | Time: {appointment2_time}")
```

.....

#task1enhanced

Use lists, dictionaries and functions to enhance the Python file

```
appointments = []
```

```
def book_appointment(patient_name, practitioner_name, appointment_time):
```

```
    if not patient_name:
```

```
        raise ValueError("Patient name cannot be empty")
```

```
    appointment = {
```

```
        "patient": patient_name,
```

```
        "practitioner": practitioner_name,
```

```
        "time": appointment_time
```

```
    }
```

```
    appointments.append(appointment)
```

```
def display_appointments():
```

```
    if not appointments:
```

```
        print("No appointments recorded.")
```

```
    return
```

```

    for appointment in appointments:
        print(f"Patient: {appointment['patient']} | Practitioner:
{appointment['practitioner']} | Time: {appointment['time']}")

print("Welcome to SmartCare: The Clinical Appointment Booking System!")
book_appointment('Alice Smith', 'Dr. John Doe', '2024-07-20 10:00 AM')
book_appointment('Bob Johnson', 'Dr. Jane Roe', '2024-07-20 11:30 AM')
display_appointments()

```

^ Now, run both programs , and identify at least five limitations.

1. Data is lost as soon as the program finishes executing.
2. Looking at both codes, the enhanced code books the appointment, but the first code only prints/displays the details rather than booking it.
3. Appointments cannot be deleted or rescheduled.
4. The python script hardcodes the values instead of asking the user's input.
5. The code allows for double bookings.

Part C - Use AI as Tutor: AI ON (Use only UC approved GenAI Tool such as Microsoft CoPilot)

Suggested prompt structure:

Act as a Python tutor.

I am learning introductory software technology.

Here is a small appointment-booking function.

1. Explain what the code does.
2. Identify three limitations.
3. Suggest improvements.
4. Do not rewrite the whole application.
5. Ask me two questions to test my understanding.

1. The program creates a simple appointment booking system using, a list to store multiple appointments, a dictionary for each appointment, and functions to book and display appoints
2. The three limitations are limited validations, no conflict checking, and date and time stored as plain text
3. Three suggestions include being able to validate all fields, prevent duplicate bookings, use python's datetime module, and add a search feature
4. n/a
5. Question 1: Why is a list used for appointments instead of a single dictionary?
Answer: A list is used because a clinic manages multiple appointments, while a single dictionary can only represent one single item's details

Question 2: Imagine a clinic receptionist books a patient's appointment but forgets to save it in the appointment system. What problem could this cause for the clinic, the practitioner, or the patient?

Answer: A problem this could cause is creating a huge risk in human error, and that it would be less effective. To mitigate this it is best to create an automated database that auto saves.

Part D - Generate an Alternative: AI ON

Ask AI to create a simple beginner-friendly Python function that stores patient name, practitioner name and appointment time. Explicitly prohibit a database or GUI.

```
# No database and no GUI are used in this program.
```

```
appointments = []
```

```
def book_appointment(patient_name, practitioner_name, appointment_time):  
    appointment = {  
        "patient": patient_name,  
        "practitioner": practitioner_name,  
        "time": appointment_time  
    }
```

```
    appointments.append(appointment)  
    print("Appointment booked successfully.")
```

```
# Example booking  
book_appointment("Alice Smith", "Dr. John Doe", "10:00 AM")
```

```
# Display stored appointments  
print(appointments)
```

Part E - Compare Human and AI Versions

Question	Human version	AI version
Easy to understand?	Yes	Yes
Runs successfully?	Yes	Yes
Uses only required features?	Yes	Yes
Adds assumptions?	Yes	Yes
Handles errors?	Yes	No
Could I explain it?	Yes	Yes

Part F - Verify Behaviour

- Normal appointment
- Blank patient name
- Two appointments for the same practitioner/time
- Strange input such as patient_name=None or appointment_time=None

Part G - Improve One Thing

Choose exactly one controlled improvement, for example: if not patient_name: raise ValueError("Patient name cannot be empty")

- My chosen improvement is to prevent blank/empty inputs from being saved as valid appointments, for example, if not patient_name or not practitioner_name or not appointment_time: print("Error:All fields are required.")

Part H - Reflection (150-250 words)

What did you build before using AI?

Before using AI, I wrote a simple Python program using lists and dictionaries to store patient appointments. My code could add an appointment and display all recorded bookings, but it was very basic. It didn't handle inputs interactively and lost all data whenever the script stopped running.

What did AI help you understand?

Using the AI as a tutor helped me understand missing error checks in my code. It pointed out edge cases I completely overlooked, like leaving the doctor's name blank or entering weird date formats.

Did AI make assumptions?

Yes, the AI made a few assumptions that were not strictly requested.

How did you verify the AI output?

I verified the AI's output by running the code in PyCharm and executing specific test cases. I intentionally entered blank strings, invalid values, and conflicting booking times to see if the script handled errors or not.

What engineering work remained for you?

The main engineering work was my responsibility because I had to choose which AI suggestions actually would've worked on the program.

Submission checklist [GitHub Commit]

- Python file runs.
- Comparison table completed.
- Normal and unusual inputs tested.
- AI assistance documented.
- Reflection completed.
- I can explain my code.